

Ejercicios Preparativos – Unidad 6 (Procedimientos y Disparadores en MySQL)

Estos ejercicios progresivos te permitirán adquirir las habilidades necesarias para desarrollar correctamente la actividad final de esta unidad.

Procedimientos (5 ejercicios)

▼ Ejercicio 1: Trasladar agentes entre oficinas

Crea un procedimiento `trasladarAgentesOficina` que reciba dos IDs de oficina. Deberá:

- Verificar que ambas oficinas existen
- Comprobar que son diferentes
- Actualizar los agentes asignados
- Mostrar un mensaje de confirmación

```
DELIMITER //
CREATE PROCEDURE trasladarAgentesOficina(IN origen INT, IN destino INT)
BEGIN
    DECLARE ex1, ex2 INT;
    DECLARE nom1, nom2 VARCHAR(50);

    SELECT COUNT(*) INTO ex1 FROM oficinas WHERE identificador = origen;
    SELECT COUNT(*) INTO ex2 FROM oficinas WHERE identificador = destino;

    IF origen = destino THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Las oficinas deben ser distintas';
    END IF;
    IF ex1 = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Oficina origen no existe';
    END IF;
    IF ex2 = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Oficina destino no existe';
    END IF;

    SELECT nombre INTO nom1 FROM oficinas WHERE identificador = origen;
    SELECT nombre INTO nom2 FROM oficinas WHERE identificador = destino;

    UPDATE agentes SET oficina = destino WHERE oficina = origen;
    SELECT CONCAT('Se han trasladado ', ROW_COUNT(), ' agentes de la oficina ', nom1, ' a ', nom2) AS mensaje;
END;
//
DELIMITER ;
```

▼ Ejercicio 2: Desactivar agentes de una familia

Actualiza todos los agentes de una familia para que su habilidad sea 0.

```
DELIMITER //
CREATE PROCEDURE desactivarAgentesFamilia(IN id_fam INT)
BEGIN
    DECLARE existe INT;
    SELECT COUNT(*) INTO existe FROM familias WHERE identificador = id_fam;

    IF existe = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Familia inexistente';
    END IF;

    UPDATE agentes SET habilidad = 0 WHERE familia = id_fam;
    SELECT ROW_COUNT() AS desactivados;
END;
//
DELIMITER ;
```

▼ Ejercicio 3: Contar agentes por oficina

Devuelve el número de agentes en una oficina específica.

```
DELIMITER //
CREATE PROCEDURE contarAgentesOficina(IN id_ofi INT)
BEGIN
    DECLARE total INT;
    SELECT COUNT(*) INTO total FROM agentes WHERE oficina = id_ofi;
    SELECT total AS total_agentes;
END;
//
DELIMITER ;
```

▼ Ejercicio 4: Validar y devolver nombre de familia

Verifica que una familia existe y devuelve su nombre, si no lanza error.

```
DELIMITER //
CREATE PROCEDURE obtenerNombreFamilia(IN id_fam INT)
BEGIN
    DECLARE nom VARCHAR(50);
    SELECT nombre INTO nom FROM familias WHERE identificador = id_fam;

    IF nom IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Familia no encontrada';
    ELSE
        SELECT nom AS nombre_familia;
    END IF;
END;
//
DELIMITER ;
```

▼ Ejercicio 5: Eliminar agentes sin oficina ni familia

Elimina agentes que no tengan asignación alguna.

```
DELIMITER //
CREATE PROCEDURE limpiarAgentesHuerfanos()
BEGIN
    DELETE FROM agentes WHERE oficina IS NULL AND familia IS NULL;
    SELECT ROW_COUNT() AS eliminados;
END;
//
DELIMITER ;
```

Triggers (5 ejercicios)

▼ Ejercicio 1: Validar nombre de agente

El nombre del agente debe tener al menos 3 caracteres.

```
DELIMITER //
CREATE TRIGGER validar_nombre
BEFORE INSERT OR UPDATE ON agentes
FOR EACH ROW
BEGIN
    IF LENGTH(NEW.nombre) < 3 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Nombre demasiado corto';
    END IF;
END;
//
DELIMITER ;
```

▼ Ejercicio 2: Validar clave segura

Impedir claves con menos de 6 caracteres.

```

DELIMITER //
CREATE TRIGGER validar_clave
BEFORE INSERT OR UPDATE ON agentes
FOR EACH ROW
BEGIN
    IF LENGTH(NEW.clave) < 6 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Clave insegura: mínimo 6 caracteres';
    END IF;
END;
//
DELIMITER ;

```

▼ Ejercicio 3: Validar horario laboral

Impedir cambios en fin de semana o fuera de horario 8-18h.

```

DELIMITER //
CREATE TRIGGER validar_jornada
BEFORE INSERT OR UPDATE OR DELETE ON agentes
FOR EACH ROW
BEGIN
    IF DAYOFWEEK(CURDATE()) IN (1,7) OR HOUR(CURTIME()) NOT BETWEEN 8 AND 18 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Fuera del horario laboral permitido';
    END IF;
END;
//
DELIMITER ;

```

▼ Ejercicio 4: Exclusividad oficina/familia

Un agente no puede tener ambas asignadas a la vez.

```

DELIMITER //
CREATE TRIGGER validar_asignacion_exclusiva
BEFORE INSERT OR UPDATE ON agentes
FOR EACH ROW
BEGIN
    IF NEW.familia IS NOT NULL AND NEW.oficina IS NOT NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Un agente no puede tener oficina y familia simultáneamente';
    END IF;
END;
//
DELIMITER ;

```

▼ Ejercicio 5: Validar agente senior

Si habilidad ≥ 8 , no puede tener familia asignada.

```

DELIMITER //
CREATE TRIGGER validar_agente_senior
BEFORE INSERT OR UPDATE ON agentes
FOR EACH ROW
BEGIN
    IF NEW.habilidad >= 8 AND NEW.familia IS NOT NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Agentes senior no deben pertenecer a familias';
    END IF;
END;
//
DELIMITER ;

```